

Intro to Object-Oriented Programming

 [_ \(https://github.com/learn-co-curriculum/python-p3-intro-to-object-oriented-programming\)](https://github.com/learn-co-curriculum/python-p3-intro-to-object-oriented-programming)  [_ \(https://github.com/learn-co-curriculum/python-p3-intro-to-object-oriented-programming/issues/new\)](https://github.com/learn-co-curriculum/python-p3-intro-to-object-oriented-programming/issues/new)

Learning Goals

- Build robust and dynamic Python objects.
- Accomplish complex programming tasks using knowledge from previous modules.

Key Vocab

- **Class**: a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
- **Initialize**: create a working copy of a class using its `__init__()` method.
- **Instance**: one specific working copy of a class. It is created when a class's `__init__()` method is called.
- **Object**: the more common name for an instance. The two can usually be used interchangeably.
- **Object-Oriented Programming**: programming that is oriented around data (made mobile and changeable in **objects**) rather than functionality. Python is an object-oriented programming language.
- **Procedural Programming**: programming that is oriented around procedures that are called in sequential order. Fortran and C are procedural programming languages.
- **Function**: a series of steps that create, transform, and move data.
- **Method**: a function that is defined inside of a class.
- **Magic Method**: a special type of method in Python that starts and ends with double underscores. These methods are called on objects under certain conditions without needing to use their names explicitly. Also called **dunder methods** (for **double underscore**).
- **Attribute**: variables that belong to an object.
- **Property**: attributes that are controlled by methods.

Introduction

Object-oriented programming (OOP) is a type of programming based on the concept of "objects", which can contain data, in the form of fields (called attributes or properties), and code, in the form of procedures (called functions or methods). OOP is all about being able to structure code so that its functionality can be shared throughout the application. This is opposed to procedural programming (PP), in which you build programs in sequential order and call procedures when you want shared behavior between pages in the application.

We'll be discussing these basics of OOP in Python:

- Classes and instances
- The `__init__()` method
- Instance methods
- Local variables and instance variables
- Object Attributes and Properties
- Setting instance variables from `__init__()`

Python is a powerful object-oriented language used by popular applications. In the upcoming lessons you'll build the foundational knowledge of OOP to be well on your way to developing your own!

Resources

- [Python documentation](https://docs.python.org/3/) ➞ [\(https://docs.python.org/3/\)](https://docs.python.org/3/)
- [Object-Oriented Programming \(OOP\) in Python 3](https://realpython.com/python3-object-oriented-programming/) ➞ [\(https://realpython.com/python3-object-oriented-programming/\)](https://realpython.com/python3-object-oriented-programming/)
- [Differences Between Procedural and Object-Oriented Programming](https://www.geeksforgeeks.org/differences-between-procedural-and-object-oriented-programming/#:%7E:text=Object%2Doriented%20programming%20is%20based,the%20concept%20of%20procedure%20abstraction.) ➞ [\(https://www.geeksforgeeks.org/differences-between-procedural-and-object-oriented-programming/#:%7E:text=Object%2Doriented%20programming%20is%20based,the%20concept%20of%20procedure%20abstraction.\)](https://www.geeksforgeeks.org/differences-between-procedural-and-object-oriented-programming/#:%7E:text=Object%2Doriented%20programming%20is%20based,the%20concept%20of%20procedure%20abstraction.)